

Estructuras de Datos

Clase 4 –

Aplicaciones de Pilas y Colas



Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Repaso: TDA Pila (Stack)

Operaciones:

- **push(e):** Inserta el elemento e en el tope de la pila
- **pop():** Elimina el elemento del tope de la pila y lo entrega como resultado. Si se aplica a una pila vacía, produce una situación de error.
- **isEmpty():** Retorna verdadero si la pila no contiene elementos y falso en caso contrario.
- **top():** Retorna el elemento del tope de la pila. Si se aplica a una pila vacía, produce una situación de error.
- **size():** Retorna un entero natural que indica cuántos elementos hay en la pila.

Código Java en archivo *Stack.java*:

```
public interface Stack<E> {  
    // Inserta item en el tope de la pila.  
    public void push( E item );  
  
    // Retorna true si la pila está vacía y falso en caso contrario.  
    public boolean isEmpty();  
  
    // Elimina el elemento del tope de la pila y lo retorna.  
    // Produce un error si la pila está vacía.  
    public E pop();  
  
    // Retorna el elemento del tope de la pila y lo retorna.  
    // Produce un error si la pila está vacía.  
    public E top();  
  
    // Retorna la cantidad de elementos de la pila.  
    public int size();  
}
```

Estructuras de datos - Dr. Sergio A. Gómez

3

Repaso: TDA Cola (Queue)

Operaciones:

- enqueue(e): Inserta el elemento *e* en el rabo de la cola
- dequeue(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía se produce un error.
- front(): Retorna el elemento del frente de la cola. Si la cola está vacía se produce un error.
- isEmpty(): Retorna verdadero si la cola no tiene elementos y falso en caso contrario
- size(): Retorna la cantidad de elementos de la cola.

4

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

```
public interface Queue<E> {  
    // Inserta el elemento e al final de la cola  
    public void enqueue(E e);  
  
    // Elimina el elemento del frente de la cola y lo retorna.  
    // Si la cola está vacía se produce un error.  
    public E dequeue();  
  
    // Retorna el elemento del frente de la cola.  
    // Si la cola está vacía se produce un error.  
    public E front();  
  
    // Retorna verdadero si la cola no tiene elementos  
    // y falso en caso contrario  
    public boolean isEmpty();  
  
    // Retorna la cantidad de elementos de la cola.  
    public int size();  
}
```

5

Combinando tipos usando genericidad

- Una cola de booleanos: Queue<Boolean>
- Una pila de colas de caracteres:
Stack<Queue<Character>>
- Una cola de pilas de colas de enteros:
Queue<Stack<Queue<Integer>>>

Estructuras de datos - Dr. Sergio A. Gómez

6

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Problema: Iteración exhaustiva

Dada una cola de pilas de caracteres, determinar cuántas veces aparece una letra c.

Se pueden destruir los datos.

```
public int contar(char c, Queue<Stack<Character>> colaPilas) {
    int cantidad = 0;
    while ( !colaPilas.isEmpty() ) {
        Stack<Character> pila = colaPilas.dequeue();
        cantidad += contarCharEnPila( c, pila );
    }
    return cantidad;
}
```

Estructuras de datos - Dr. Sergio A. Gómez

7

Problema: Iteración exhaustiva

Dada una cola de pilas caracteres: Determinar cuántas veces aparece una letra c. Se pueden destruir los datos.

```
public int contarCharEnPila(char c, Stack<Character> pila) {
    int cantidad = 0;
    while ( !pila.isEmpty() ) {
        char tope = pila.top();
        if ( tope == c ) cantidad ++;
        pila.pop();
    }
    return cantidad;
}
```

Estructuras de datos - Dr. Sergio A. Gómez

8

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Problema: Iteración no exhaustiva

Determinar si un ítem de dato está en una pila.

Hay que vaciar la pila buscando el dato.

Notar que la iteración termina cuando la pila está vacía y se determina que no está.

O la iteración termina cuando encontré el dato y determino que sí está.

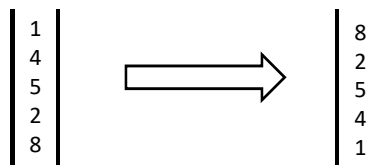
Notar que el método es genérico. Eso se indica con <E>:

```
public <E> boolean contiene(Stack<E> pila, E item) {
    boolean encuentre = false;
    while ( !pila.isEmpty() && !encuentre ) {
        if ( pila.top().equals(item) ) encuentre = true;
        else pila.pop();
    }
    return encuentre;
}
```

Estructuras de datos - Dr. Sergio A. Gómez

9

Invertir el contenido de una pila

Opción de implementación:

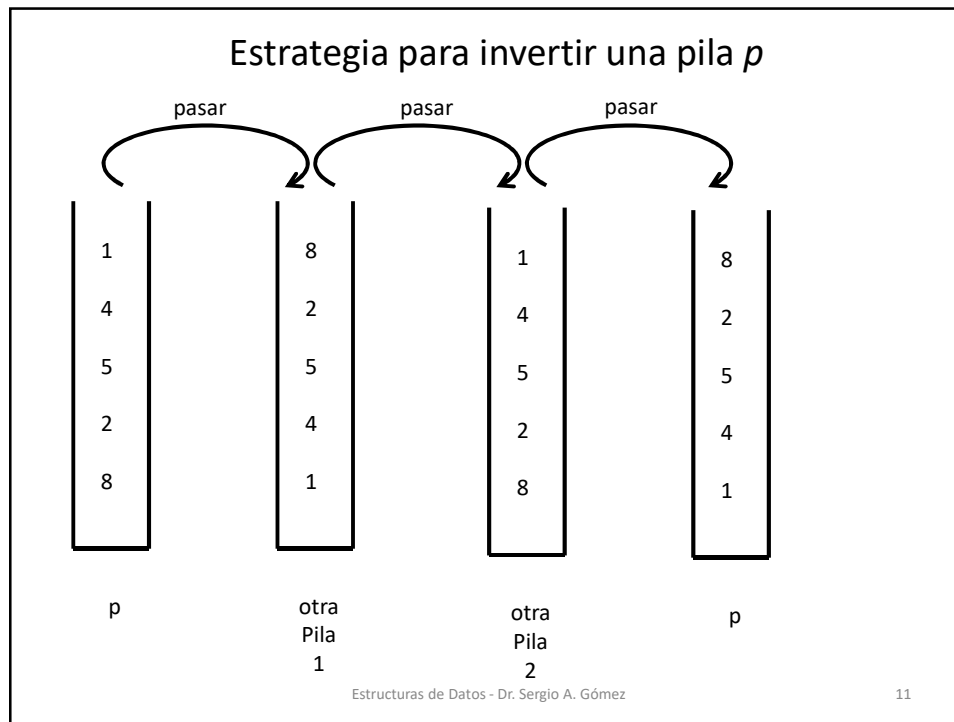
Invertir el contenido de una pila es un algoritmo cuya estructura es independiente del tipo de los elementos de la pila.

Escribir una clase *InvertidorDePila* que reciba una pila genérica y retorne la misma pila pero con su contenido invertido.

Estructuras de datos - Dr. Sergio A. Gómez

10

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.



Invertir pila: Solución

```
import datos.pila.Stack; // Importo nuestra interfaz de pila
import datos.pila.PilaArreglo; // Importo nuestra implementación de pila

public class Principal {
    public static void main( String [] args )
    {
        ...
        Stack<Character> p = new PilaArreglo<Character>();
        p.push( 'a' ); p.push( 'b' ); p.push( 'c' );

        InvertidorDePila inv = new InvertidorDePila();
        inv.invertirPila( p );
        ...
    }
}
```

Estructuras de datos - Dr. Sergio A. Gómez

12

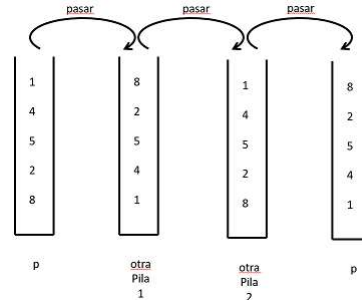
El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Invertir Pila genérica

```
import datos.pila.Stack; // Importo nuestra interfaz de pila
import datos.pila.PilaArreglo; // Importo nuestra implementación de pila
public class InvertidorDePila {
    // Pasa el contenido de pila p1 a pila p2.
    private <E> void pasar( Stack<E> p1, Stack<E> p2 ) {
        while ( !p1.empty() )
            p2.push(p1.pop());
    }

    // Invierte el contenido de la pila p.
    public <E> void invertirPila(Stack<E> p) {

        Stack<E> otraPila1, otraPila2;
        otraPila1 = new PilaArreglo<E>();
        otraPila2 = new PilaArreglo<E>();
        pasar( p, otraPila1 );
        pasar( otraPila1, otraPila2 );
        pasar( otraPila2, p );
    }
}
```



13

Aplicaciones de Pilas y Colas

- Las colas sirven para construir secuencias que deben ser procesadas en el orden en que fueron generadas mirando exactamente una vez a cada elemento.
- Las pilas sirven para procesar elementos en el orden opuesto en el que fueron procesadas, lo cual habilita a testear simetría en secuencias.

Problemas clásicos con pilas y colas

- Paréntesis anidados
- Evaluación de expresiones polaca inversa
- Validación de código HTML

Estructuras de datos - Dr. Sergio A. Gómez

15

Procesamiento de Paréntesis

Nos interesa determinar si en una expresión aritmética los símbolos de agrupación coinciden.

Los símbolos de agrupación son:

- Paréntesis: (y)
- Llaves: { y }
- Corchetes: [y]

Cada símbolo de apertura debe coincidir su símbolo de cierre.

Ejemplos:

1. Correcto: [(5 + x) - (y - z)]
2. Correcto: () () { { ({ }) } }
3. Correcto: [[() (())]] { () }
4. Incorrecto:) () []
5. Incorrecto: ([]
6. Incorrecto: (([

Estructuras de Datos - Dr. Sergio A. Gómez

16

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Algoritmo CoincidenParentesis(Q)

Entrada: Una cola de Q de tokens, cada uno es un símbolo de agrupación, una variable, un operador aritmético o un número

Salida: true si y solo si los paréntesis de Q coinciden

Sea S una pila vacía

Ok $\leftarrow$ true

while Q no está vacía **and** ok **do**

 x $\leftarrow$ Q.dequeue()

if x es un símbolo de agrupación de apertura **then**

 S.push(x)

else if x es un símbolo de agrupación de cierre **then**

if S.isEmpty() **then**

 ok $\leftarrow$ false

 { No hay nada con qué coincidir. }

else if S.pop() no coincide con el tipo de x **then**

 ok $\leftarrow$ false

 { Encontré un tipo incorrecto de símbolo de cierre. }

if ok **and** S.isEmpty() **then** { Cada símbolo de cierre coincidió con el de apertura y no sobra nada. }

return true

else return false

 { Algo falló antes o algunos símbolos de apertura nunca

 fueron cerrados. }

Casos de prueba:

1. Correcto: [(5 + x) - (y - z)]

2. Correcto: () () { { { } } }

3. Correcto: [[() (())]] { () }

4. Incorrecto:) () []

5. Incorrecto: ([]

6. Incorrecto: (([]

Estructuras de Datos - Dr. Sergio A. Gómez

17

Evaluación de expresiones

- Evaluaremos expresiones en notación polaca inversa

Estructuras de datos - Dr. Sergio A. Gómez

18

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Ejemplos de equivalencias entre las notaciones

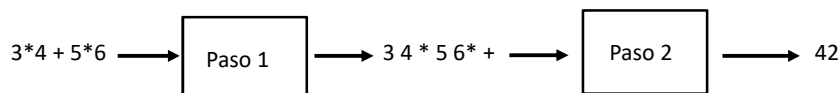
Infija	Postfija (Polaca inversa)
3	3
3 + 4	3 4 +
7 * 5	7 5 *
3 + 4 * 5	3 4 5 * +
4 * 5 + 3	4 5 * 3 +
3 + 4 + 1	3 4 + 1 +
3 * 4 + 5 * 6	3 4 * 5 6 * +

Estructuras de datos - Dr. Sergio A. Gómez

19

Evaluación de expresiones:
esquema generalSolución:

- Entrada: Expresión aritmética infija válida
con operadores +, -, *, / binarios
con * y / con mayor precedencia que + y -
y sin paréntesis
- Paso 1: Traducir expresión infija en postfija (está en el apéndice).
- Paso 2: Evaluar expresión postfija

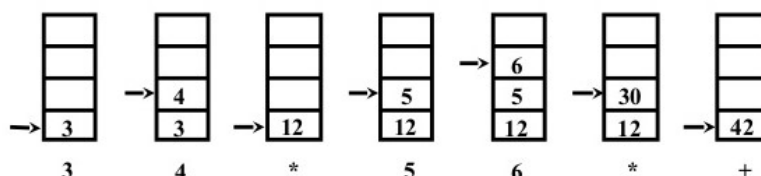


20

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Paso 2: Cómo evaluar expresiones en notación polaca inversa (postfija)

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * + \rightarrow 42$



Dada una expresión en notación postfija, se comienza con una pila vacía. Se consumen los símbolos de a uno por vez. Los operandos se apilan. Con cada operador se desapilan dos operandos, se aplica el operador y se vuelve a apilar el resultado. Si al finalizar la expresión, la pila tiene un elemento, ese elemento es el resultado de la expresión. Si la pila tiene más de un elemento, la expresión estaba incorrectamente formada. Si al tratar de aplicar un operador no hay por lo menos dos elementos en la pila, la expresión está mal formada.

Estructuras de datos - Dr. Sergio A. Gómez

21

Paso 2: Evaluación de expresión válida en notación postfija

$P \leftarrow \text{new Pila}()$

Repetir

átomo $\leftarrow E.\text{dequeue}()$

si átomo es operando entonces

$P.\text{push}(\text{átomo})$

sino // átomo es un operador

operador $\leftarrow \text{átomo}$

operando2 $\leftarrow P.\text{pop}()$

operando1 $\leftarrow P.\text{pop}()$

resultado $\leftarrow \text{operar}(\text{operador}, \text{operando1}, \text{operando2})$

$P.\text{push}(\text{resultado})$

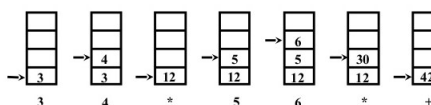
Hasta $E.\text{empty}()$

Resultado $\leftarrow P.\text{pop}()$

Retornar resultado

Nota: operando2 sale primero en caso en que el operador no sea conmutativo (como $-$ y $/$)

$(3 * 4) + (5 * 6) \Rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

22

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Problema

- Validación de archivo HTML.

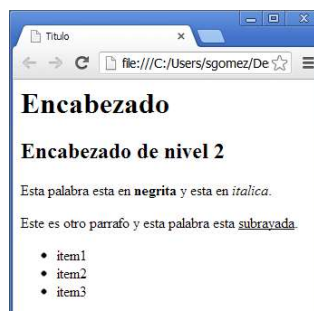
Estructuras de datos - Dr. Sergio A. Gómez

23

Validación de código HTML

- Página web:** Archivo de texto con extensión HTML conteniendo texto con marcas (*tagged text*) y texto sin marcas (*untagged text*).
- Un tal archivo es interpretado por un navegador web y el texto sin marcas es formateado de acuerdo al texto con marcas para su presentación en la pantalla del navegador.

```
<html>
<head><title>Titulo</title></head>
<body>
  <h1>Encabezado</h1>
  <h2>Encabezado de nivel 2</h2>
  <p>Esta palabra esta en <b>negrita</b>
  y esta en <i>italica</i>.</p>
  <p>Este es otro parrafo
  y esta palabra esta <u>subrayada</u>.</p>
  <ul>
    <li>item1</li>
    <li>item2</li>
    <li>item3</li>
  </ul>
</body>
</html>
```



Estructuras de datos - Dr. Sergio A. Gómez

24

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Validación de código HTML

Marcas:

- De apertura: <html>, <head>, <p>, ...
- De cierre: </html>, </head>, </p>, ...

No consideraremos marcas con atributos:

Por ej.: <p align="right">

No consideraremos marcas únicas:

Por ej.:

```
<html>
<head><title>Titulo</title></head>
<body>
  <h1>Encabezado</h1>
  <h2>Encabezado de nivel 2</h2>
  <p>Esta palabra esta en <b>negrita</b>
  y esta en <i>italica</i>.</p>
  <p>Este es otro parrafo
  y esta palabra esta <u>subrayada</u>.</p>
  <ul>
    <li>item1</li>
    <li>item2</li>
    <li>item3</li>
  </ul>
</body>
</html>
```

align es un atributo de la
marca p con valor right

En las marcas únicas, la marca
se abre y se cierra en la misma

Estructuras de datos - Dr. Sergio A. Gómez

25

Mini curso de HTML

- Página HTML = <html>Encabezado Cuerpo</html>
- Encabezado = <head> </head>
- Cuerpo = <body> ... </body>
- Contenido del encabezado: <title>...</title> (otras marcas incluyen detalle del idioma, palabras clave para los buscadores como Google que no consideraremos)

```
<html>
<head><title>Titulo</title></head>
<body>
  <h1>Encabezado</h1>
  <h2>Encabezado de nivel 2</h2>
  <p>Esta palabra esta en <b>negrita</b>
  y esta en <i>italica</i>.</p>
  <p>Este es otro parrafo
  y esta palabra esta <u>subrayada</u>.</p>
  <ul>
    <li>item1</li>
    <li>item2</li>
    <li>item3</li>
  </ul>
</body>
</html>
```

Estructuras de datos - Dr. Sergio A. Gómez

26

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
"Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Mini curso de HTML

Encabezados:

`<h1>Pilas</h1>`

`<h2>Implementaciones</h2>`

`<h3> Con arreglos</h3>`

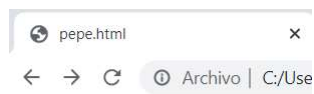
`<h3> Con nodos enlazados</h3>`

`<h1>Colas</h1>`

`<h2>Implementaciones</h2>`

`<h3> Con arreglo circular</h3>`

`<h3> Con nodos enlazados</h3>`



Pilas

Implementaciones

Con arreglos

Con nodos enlazados

Colas

Implementaciones

Con arreglo circular

Con nodos enlazados

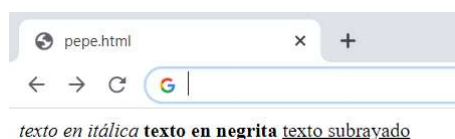
Estructuras de datos - Dr. Sergio A. Gómez

27

Mini curso de HTML

Formato de texto:

- Cursiva (italics): `<i>texto en itálica</i>`
- Negrita (bold): `<b>texto en negrita</b>`
- Subrayado (underlined): `<u>texto subrayado</u>`



Estructuras de datos - Dr. Sergio A. Gómez

28

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

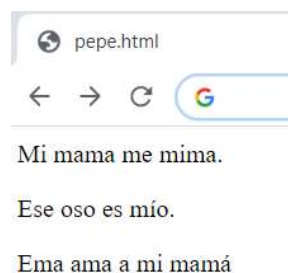
Mini curso de HTML

Párrafos: `<p>Contenido del párrafo.</p>`

`<p>Mi mama me mima.</p>`

`<p>Ese oso es mío.</p>`

`<p>Ema ama a mi mamá</p>`



Estructuras de datos - Dr. Sergio A. Gómez

29

Mini curso de HTML

Listas con viñetas (ul = unordered list, li = list item):

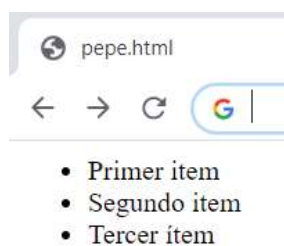
`<ul>`

`<li>Primer item</li>`

`<li>Segundo item</li>`

`<li>Tercer ítem</li>`

`</ul>`



Estructuras de datos - Dr. Sergio A. Gómez

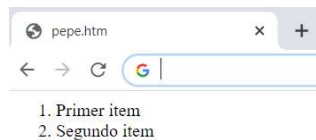
30

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Mini curso de HTML

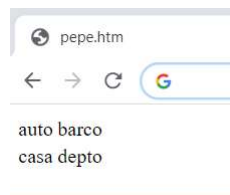
Listas con enumeraciones (ordered list):

```
<ol>
  <li>Primer item</li>
  <li>Segundo item</li>
</ol>
```



Tablas: (table = tabla, tr = table row, td = table datum)

```
<table>
<tr>
  <td>auto</td>
  <td>barco</td>
</tr>
<tr>
  <td>casa</td>
  <td>depto</td>
</tr>
</table>
```



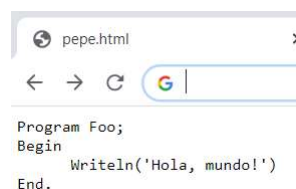
Estructuras de datos - Dr. Sergio A. Gómez

31

Mini curso de HTML

Texto verbatim:

```
<pre>
Program Foo;
Begin
  Writeln('Hola, mundo!')
End.
</pre>
```



Estructuras de datos - Dr. Sergio A. Gómez

32

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

A HTML no le interesan los blancos

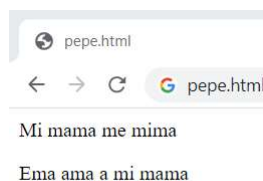
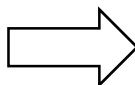
```
<p>Mi mama me mima</p>
```

```
<p>Ema
```

```
ama a
```

```
mi
```

```
mama</p>
```



En la visualización, 2 o más blancos equivalen a 1 blanco (idem TAB, CR, LF).

Paso 1: Tomar la lista de tokens a partir del archivo de texto HTML.

```
<html>
<head><title>Titulo</title></head>
<body>
  <h1>Encabezado</h1>
  <h2>Encabezado de nivel 2</h2>
  <p>Esta palabra esta en <b>negrita</b>
  y esta en <i>italica</i>.</p>
  <p>Este es otro parrafo
  y esta palabra esta <u>subrayada</u>.</p>
  <ul>
    <li>item1</li>
    <li>item2</li>
    <li>item3</li>
  </ul>
</body>
</html>
```

Cola de tokens: "<html>", "<head>", "<title>", "Titulo", "</title>", "</head>", "<body>", "<h1>", "Encabezado", "</h1>", "<h2>", "Encabezado", "de", "nivel", "2", "</h2>", ...

IDEA DE CÓMO PROCESAR EL ARCHIVO DE TEXTO HTML PARA GENERAR LA COLA DE TOKENS

```
import java.io.*;

public Queue<String> recuperarColaTokens(String archivoHTML) throws Exception {

    File file = new File( archivoHTML );
    BufferedReader br = new BufferedReader(new FileReader(file));
    Queue<String> colaTokens = new ColaArregloCircular<String>();
    String linea = br.readLine();
    while( linea != null ) {
        partirLineaYEncolarTokens(linea, colaTokens);
        linea = br.readLine();
    }
    return colaTokens;
}
```

Estructuras de Datos - Dr. Sergio A. Gómez

35

Recorrer la línea y cuando encuentro un <, recorro hasta encontrar un > acumulando en token todo lo que vi. Al llegar a >, encolo el token en la cola y preparo un token vacío para la próxima.

```
public void partirLineaYEncolarTokens(String linea, Queue<String> q) {
    String token = "";
    int i=0;
    while( i < linea.length() ) {
        if (linea.charAt(i) == '<') {
            char c = linea.charAt(i++);
            token += c;
            do {
                c = linea.charAt(i++);
                token += c;
            } while ( c != '>');
            q.add( token );
            token = "";
        } else {
            i++;
        }
    }
}
```

Estructuras de Datos - Dr. Sergio A. Gómez

36

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Algoritmo Validar Archivo HTML

Entrada: Archivo HTML H

Salida: true si el archivo es válido y false en caso contrario

PilaTags $\leftarrow$ new Pila<String>()

ColaTokens $\leftarrow$ recuperarColaTokens(H)

EsValido $\leftarrow$ true

Mientras NOT ColaTokens.isEmpty() AND EsValido

 Token $\leftarrow$ ColaTokens.dequeue()

 si Token es una marca de apertura entonces

 PilaTags.push(Token)

 sino

 si Token es una marca de cierre entonces

 Si NOT PilaTags.isEmpty() AND Token coincide con PilaTags.top() entonces

 PilaTags.pop()

 sino

 EsValido $\leftarrow$ false

Si NOT PilaTags.isEmpty() entonces

 EsValido $\leftarrow$ false

Retornar EsValido

```
<html>
<head><title>Titulo</title></head>
<body>
  <h1>Encabezado</h1>
  <h2>Encabezado de nivel 2</h2>
  <p>Esta palabra esta en <b>negrita</b>
  y esta en <i>italica</i>.</p>
  <p>Este es otro parrafo
  y esta palabra esta <u>subrayada</u>.</p>
  <ul>
    <li>item1</li>
    <li>item2</li>
    <li>item3</li>
  </ul>
</body>
</html>
```

<title>	</title>
<head>	Como es un tag
<html>	de cierre
+-----+	tengo que ver si
PilaTags	coincide con el tope

37

Bibliografía

- Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th edition, John Wiley & Sons, 2006, Capítulo 5

38

Apéndice

- Conversión de expresiones en notación infija a postfija.
- Este algoritmo se atribuye a Donald Knuth.

Donald Knuth 77 idiomas

Artículo Discusión Leer Editar Ver historial Herramientas

Donald Ervin Knuth (Milwaukee, Wisconsin; 10 de enero de 1938) es un reconocido experto en ciencias de la computación estadounidense y matemático, famoso por su fructífera investigación dentro del análisis de algoritmos y compiladores.¹

Es Profesor Emérito de la Universidad de Stanford.²

Biografía [editar]

Primeros años [editar]

Knuth nació en Milwaukee, Wisconsin, hijo de Ervin Henry Knuth y Louise Marie Bohning.³ Describe su herencia como «alemana luterana del Medio Oeste».⁴ ⁶⁶ Su padre tenía una pequeña imprenta y enseñaba contabilidad.⁵

Información personal

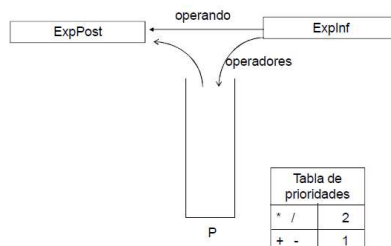
Nombre en inglés Donald Ervin Knuth ✓

Estructuras de datos - Dr. Sergio A. Gómez

39

Paso 1: Paso de notación infija a postfija

- **Entrada:** *ExpInf* es una cola que contiene una expresión válida en notación infija, sin parentizar y con operadores aditivos (+, -) y multiplicativos (*, /).
- **Salida:** *ExpPost* es una cola que al finalizar queda con la expresión en notación postfija
- Se usa una pila auxiliar de operadores según el esquema que se detalla abajo.
- Luego de retirar el último elemento de la cola *ExpInf*, se desapila todo lo que resta y se lo ingresa en la cola *ExpPost*.



40

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente: "Estructuras de Datos. Notas de Clase". Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

Paso 1: Paso de notación infija a postfija

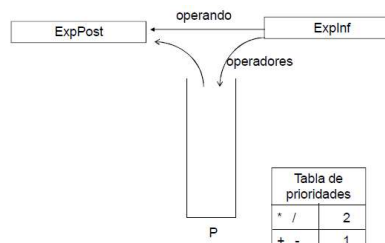
$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino
        // átomo es un operador
        Mientras (no P.isEmpty()) Y (prioridad(átomo) <= prioridad(P.top() ) )
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Estructuras de datos - Dr. Sergio A. Gómez

41

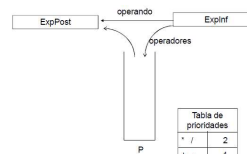
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino
        // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top() ) )
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

Estructuras de datos - Dr. Sergio A. Gómez

42

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

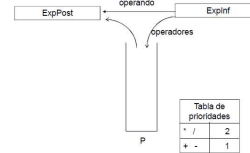
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

ExpPost



$3 * 4 + 5 * 6$

Explnf

Estructuras de datos - Dr. Sergio A. Gómez

43

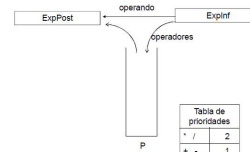
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

3

ExpPost



$* 4 + 5 * 6$

Explnf

Estructuras de datos - Dr. Sergio A. Gómez

44

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

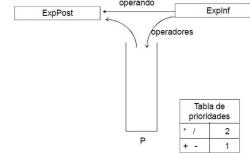
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

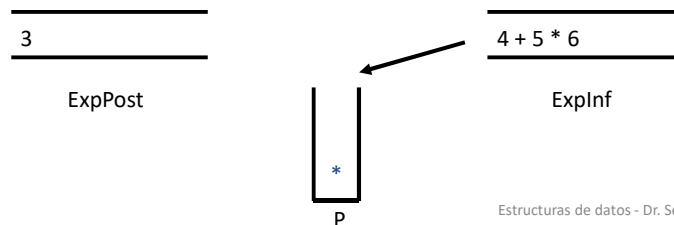
Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

45

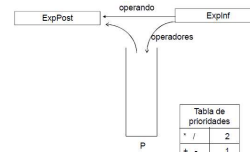
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

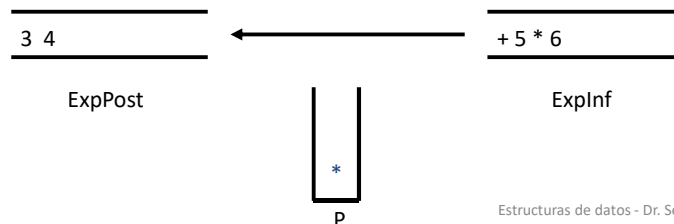
Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

46

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

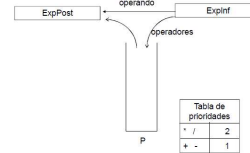
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

3 4 *

ExpPost

+ 5 * 6

Explnf



Estructuras de datos - Dr. Sergio A. Gómez

47

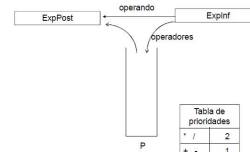
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

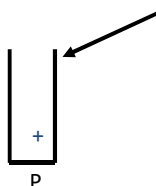
$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

3 4 *

ExpPost

5 * 6

Explnf



Estructuras de datos - Dr. Sergio A. Gómez

48

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

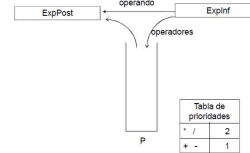
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

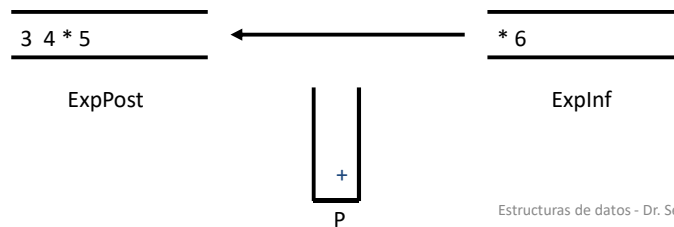
Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

49

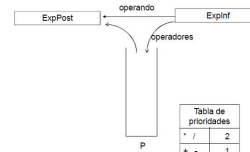
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

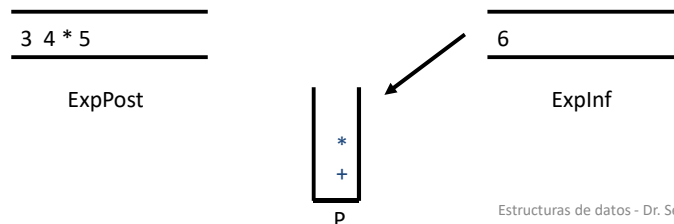
Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

50

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

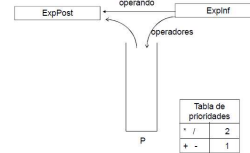
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

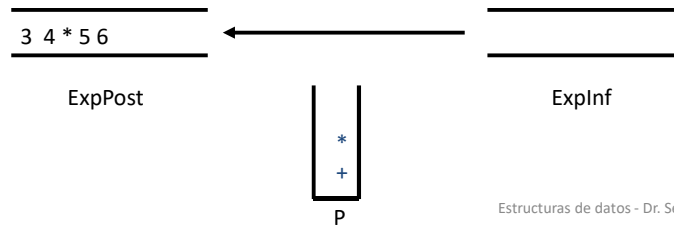
Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

51

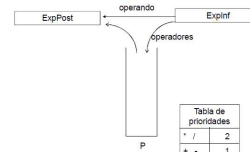
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

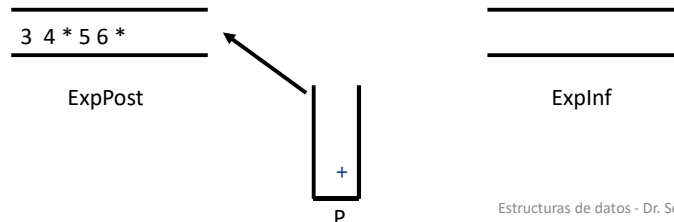
Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$



Estructuras de datos - Dr. Sergio A. Gómez

52

El uso total o parcial de este material está permitido siempre que se haga mención explícita de su fuente:
 “Estructuras de Datos. Notas de Clase”. Sergio A. Gómez. Universidad Nacional del Sur. (c) 2013-2025.

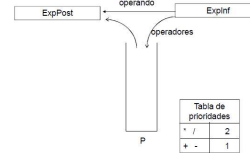
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

3 4 * 5 6 * +

ExpPost



Explnf

Estructuras de datos - Dr. Sergio A. Gómez

53

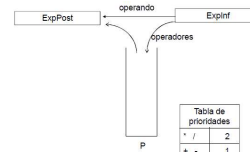
Paso 1: Paso de notación infija a postfija

```

ExpPost ← new Cola()
P ← new Pila()
Repetir
    átomo ← Explnf.dequeue()
    si átomo es operando entonces
        ExpPost.enqueue( átomo )
    sino // átomo es un operador
        Mientras (no P.isEmpty()) Y
            (prioridad(átomo) <= prioridad(P.top()))
            ExpPost.enqueue( P.pop() )
        P.push( átomo )
Hasta Explnf.isEmpty()

Mientras no P.isEmpty()
    ExpPost.enqueue( P.pop() )

```



Caso de prueba:

$3 * 4 + 5 * 6 \rightarrow 3 4 * 5 6 * +$

3 4 * 5 6 * +

ExpPost



Explnf

Ok!

Estructuras de datos - Dr. Sergio A. Gómez

54